

コンピュータアーキテクチャI

担当：佐々木 敬泰

第2回

コンピュータの構成

シラバス

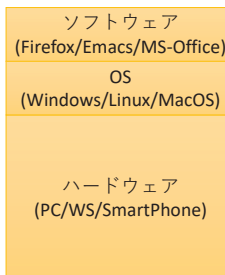
1. イントロダクション
2. コンピュータの構成
3. 数の表現
4. 様々なプロセッサと命令セット
5. 基本命令
6. メモリアクセス命令
7. 分岐命令
8. 高級言語からアセンブラへの変換
9. プログラムの翻訳と起動
10. 加減算
11. 乗算アルゴリズムと乗算器
12. 除算アルゴリズムと除算器
13. 浮動小数点
14. 浮動小数点の乗算と誤差
15. 定期試験

今日の内容

- コンピュータの概要
- 単位系の整理
- 8つの主要なアイデア
- プログラムの裏側
- コンピュータの内部
- 製造技術
- コンピュータの性能

コンピュータの概要

- 現代の生活に欠かせないコンピュータ
 - 動作原理は？
 - どうやって作る？
- ⇒コンピュータの基礎から学習する



コンピュータの抽象化とテクノロジー

コンピュータを理解するための要素

- 単位系
- 8つの主要なアイデア
- プログラムの裏側
- コンピュータの内部
- コンピュータの性能

単位系

CPUの動作速度、メモリや記憶装置の容量を含め、年々増加している。そこで、まずはコンピュータの構成や性能を表す単位系について簡単にまとめる。

単位

- 性能等を表す単位

単位	読み	意味
b (bit)	ビット	情報量の最小単位。2進数の一桁。
B (byte)	バイト	情報量の単位。通常は1byte=8bit。
o(octet)	オクテット	(厳密に) 8bit

- コンピュータの中のデータは基本的にすべて2進数で扱う

語長(1/2)

- コンピュータが一度に扱える数値の大きさの単位を「語(word)」と呼ぶ(不正確)
- 語長はアーキテクチャに依存
 - 8bit
 - PICマイコン、AVRマイコン
 - 16bit
 - Intel 8086、ルネサス78K、日立H8 (一部)
 - 32bit
 - Intel 80386、MIPS R3000、ARMv7
 - 64bit
 - Intel Core2以降、MIPS64、ARM64

ファミコン

スーファミ

Wii/WiiU

NINTENDO64

語長(2/2)

- 64bit CPUでは、必ずしも1word=8Bとは限らない
 - DEC Alpha
 - Longword = 4B
 - Quadword = 8B
 - RISC-V (RV64I)
 - LD命令は8B(64bit)読み出し
 - LW命令は4B(32bit)読み出し
- 64bit CPUの場合は32bit CPUとの互換性の関係でいろいろ変則的
- 本講義では基本的には32bit CPUを想定

語、半語、バイト

- MIPS (32bitプロセッサ) の場合
 - 1語 = 1word = 4バイト(B) = 32bit
 - 半語 = 1 half word = 2バイト(B) = 16bit
 - 1バイト(B) = 8bit (0~255)
- 8bit CPUは200+300ができない??
 - 1命令ではできない
 - コンパイラが複数の加算命令に分解
 - ⇒乗算・除算もコンパイラが頑張る

例) 東芝のとある500GB HDD

フォームファクター	2.5型
インターフェース	SATA 6 Gbit/s
フォーマット	512e
バッファサイズ	16 MiB
ディスク回転数	7278 rpm
記憶容量	500GB



記憶容量: 1GB = 1,000,000,000 (10の9乗) バイトによる算出値です。しかし、1GB = 1,073,741,824 (2の30乗) バイトによる算出値をハードディスク容量として用いるコンピュータオペレーティングシステムでは、記載よりも少ない容量がハードディスク容量として表示されます。

⇒容量は466GiB。HDD業界の慣習に基づき500GBと表記。

2のべき乗か10のべき乗が注意する必要がある!

8つの主要なアイディア

コンピュータが発明されて以来、様々な技術によりコンピュータは進歩してきた。そこで、コンピュータアーキテクチャの改良に寄与してきた主要な8つのアイディアについて簡単にまとめる。

8つの主要なアイディア

- SDGsのような何か...



Mooreの法則



抽象化



一般的な場合を高速化



並列処理



パイプライン処理



予測



階層



信頼性

ムーアの法則

何の根拠もない!



Gordon E. Moore
Intel創業メンバーの1人
(出典: Intel)

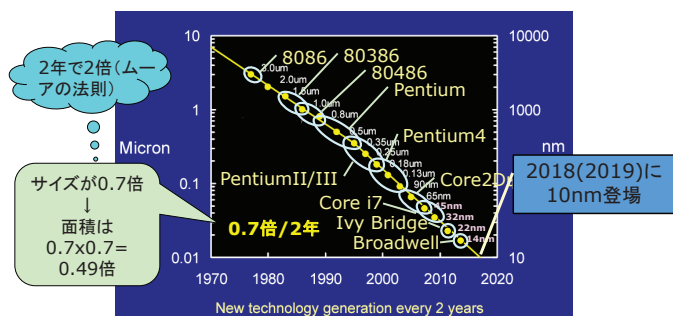
半導体チップの集積度は、およそ18~24カ月で2倍になる(1965)



未だにこの法則に乗って進歩してる

微細化の進歩

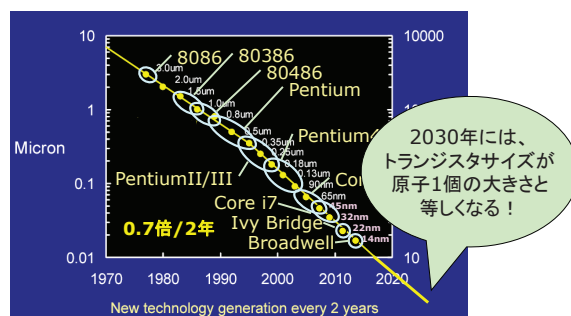
- 集積技術は年々向上



出典: 2004 Topical Research Conference on Reliability, Intel

微細化の進歩

- 集積技術は年々向上



出典: 2004 Topical Research Conference on Reliability, Intel

抽象化

- ムーアの法則に従い、チップ当たりのトランジスタ数が劇的に増加した（2016年発売のCore i7 6950Xで32億個）
- それにより、プロセッサは複雑化し、設計の長期化を引き起こした
- 従って、ハードウェア・ソフトウェアとも、抽象化し、層別設計することでモデルの単純化をする必要がある
 - ⇒全体を細かく見渡せるレベルではないため

一般的な場合を高速化(1/2)

- 稀な場合を高速化するよりも、一般的な場合を高速化の方が性能向上につながりやすい
- 例)


```
sum=0;
for(i=0; i<10000; i=i+1){
    sum = sum + price[i];
}
pay = sum * 1.1; /* 消費税 */
```

$$sum = \sum_{i=0}^{9999} price_i$$

$$pay = sum \cdot 1.1$$
- 足し算と掛け算、どちらを高速化すべきか？

一般的な場合を高速化(2/2)

- AMDのブルドーザアーキテクチャ
 - CPUには整数演算器と浮動小数点演算器がある
 - ゲーム等では浮動小数点演算は頻発するが、サーバ（WEBサーバやメールサーバ等）では浮動小数点演算は少ない
- ⇒整数演算器を増強
- ⇒浮動小数点演算器は減少

並列処理

- ムーアの法則に頼った性能向上は限界（の兆しが見えてる）
- 逐次処理には限界がある
 - 1THzのCPUは（ほぼ）実現不能
- 処理を分割し、並列に行う
 - 並列計算機
 - マルチコアプロセッサ
 - メニーコアプロセッサ
 - (GP)GPU

パイプライン処理

- 並列処理の一種
 - 流れ作業で高速化する
- 例) 目玉焼きを100皿作る

逐次処理
は遅い



パイプライン処理

- 並列処理の一種
 - 流れ作業で高速化する
- 例) 目玉焼きを100皿作る

3倍速く
なる



予測

- 結果を予測して高速化
 - 「投機的実行」と呼ぶ技術
 - 分岐予測
 - 例)

俺、この戦いが終わったら、結婚するんだ

⇒前もって指輪を購入
 - 値予測
 - 例)

前回一位だったからトウカイテイオーに100枚

投機的実行

【プログラム】

```
A = 3.14
B = sin(60*A/180)
IF B > 0.5 THEN
    D = A * 2
ELSE
    D = A - 3
```

投機的実行

計算に時間が掛かる(答えは約0.87)

多分B>0.5になる(予測)

Bの計算終了後に予測成功とわかった
⇒投機的実行成功
Bの計算終了後に予測失敗とわかった
⇒投機的実行失敗

- Dの計算結果を破棄
- 演算器を元の状態に戻す
- IF文から再実行



値予測

- 計算に時間の掛かる処理の結果を予測して先の処理を進める
 - 例)


```
x=sin(0.0)*sin(0.1)*sin(0.2)*sin(0.3)*sin(0.4)*sin(0.5)*sin(0.6)*sin(0.7)*sin(0.8)*sin(0.9);
y=x+2;
```

xの値を予測(多分0)して、yの値を先に計算する。
もし、xが0じゃない場合は、y=x+2を再計算する。
⇒後続命令に**副作用**がないことが条件



副作用

- コンピュータの世界で重要な概念
 - コンピュータの論理的状态を変化させる作用
 - グローバル変数の変更
 - ファイルの読み書き
 - 画面の表示
 - 他、いろいろ
- ⇒大雑把に言えば、あとで取り返しのつかない処理

```
x=時間の掛かる関数();
y=x+2;
値を画面に出力する関数(y);
```



階層

- 抽象化と関連して、処理や手続き、回路等を層に分割し、軽量化・高速化・効率化する
 - 例)
 - 記憶階層
 - メモリの高速化は高性能化に繋がる
 - しかし、高速なメモリは高価
- ⇒小容量の高速なメモリと大容量の安価なメモリを組み合わせることで記憶システムを作る



信頼性

- コンピュータには高速性や低消費電力性に加え、信頼性も求められる
- 一般的には、「冗長性」を持たせることで信頼性を向上させる
 - メモリのECC追加
 - HDD/SSDのRAID化
 - プロセッサの冗長化



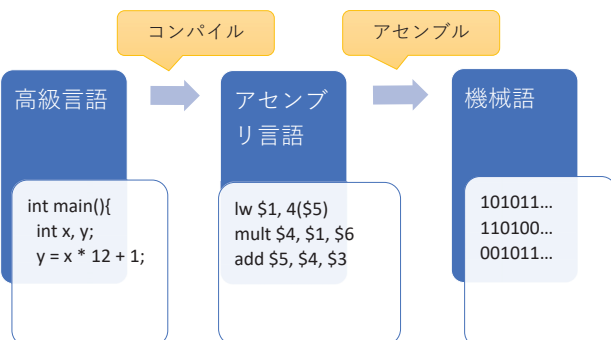
プログラムの裏側

コンパイラ

- コンピュータが実行できるのは機械語のプログラムのみ
 - 人間が機械語を書くのは大変
 - アセンブラでアセンブリ言語から機械語に変換する
 - アセンブリ言語も人間にとっては大変
 - 高級言語からアセンブリ言語に翻訳
 - ↑「コンパイル」と呼ぶ



コンパイルの流れ



高級言語からアセンブリ言語へ

- C言語のプログラム例(抜粋)


```
int x, y, z;
x = 10;
y = 20;
z = x + y;
```
- アセンブリ言語


```
addi $4, $0, 10
addi $5, $0, 20
add $4, $4, $5
sw $4, 0($3)
```

アセンブリ言語へから機械語へ

• アセンブリ言語

```
addi $4, $0, 10
addi $5, $0, 20
add $4, $4, $5
sw $4, 0($3)
```

• 機械語

```
• 010100111.....
• 101000101.....
• 110100010.....
• 001011001.....
```

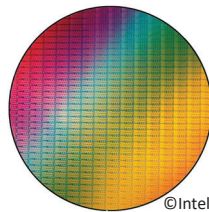
基本的には
1対1対応

コンピュータの内部

プロセッサおよびメモリを
製造するための技術

半導体製造技術

- 現在のプロセッサやメモリは膨大な数のトランジスタ(MOS-FET)で実装されている（ムーアの法則）
- 8～12インチ（20～30cm）くらいのシリコンウェハに写真・印刷技術を応用した手法で回路を形成する



製造・テスト・コスト

- 製造したチップの中には欠陥を含んだ不良品(defect)が出る
⇒製造したチップはすべてテストして欠陥品を排除する
- 良品の率を「歩留まり(yield)」と呼ぶ（歩留まりは各社とも極秘扱いが多い）
- ダイ当たりのコスト = $\frac{\text{ウェハ当たりのコスト}}{\text{ウェハ当たりのダイの数} \times \text{歩留まり}}$
- 歩留まり = $\frac{1}{1 + (\text{単位面積の欠陥数} \times \text{ダイの面積} / 2)}^2$. . . （経験則）

コンピュータの性能

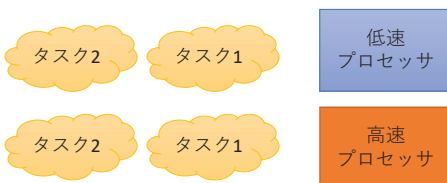
性能が良いコンピュータとは？

性能

- コンピュータの性能#とは？
 - 応答速度（実行時間）
 - ユーザ視点：作業を開始してから終了するまでの時間
 - スループット（バンド幅）
 - 管理者視点：一定時間内に終了した作業の総量（あるいは単位時間当たりの作業量）
 - 他にも、
 - コスト（価格）
 - 消費電力
 - 設置面積
 - 重さ
 - 等々、使用条件により様々ある。

応答速度とスループットの関係(1/2)

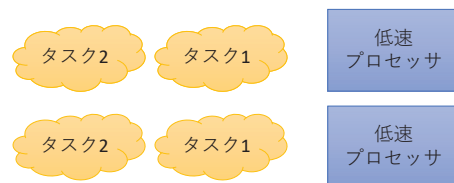
• 高速なプロセッサに置き換える



- 低速プロセッサ
 - 応答時間は2秒/タスク
 - スループットは0.5個/秒
 - 高速プロセッサ
 - 応答時間は1秒/タスク
 - スループットは1個/秒
- 応答時間、スループット共に向上

応答速度とスループットの関係(2/2)

• プロセッサを複数用意する



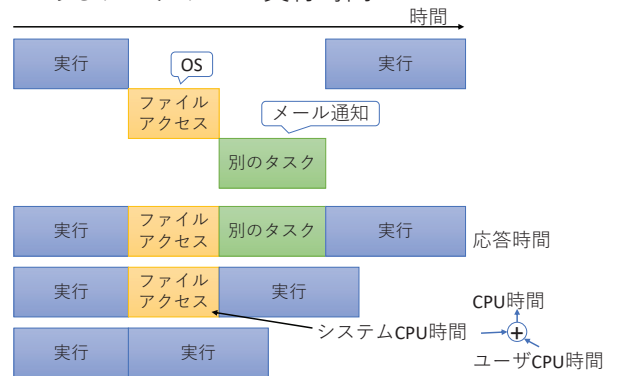
- シングルプロセッサ
 - 応答時間は2秒/タスク
 - スループットは0.5個/秒
 - デュアルプロセッサ
 - 応答時間は2秒/タスク
 - スループットは1個/秒
- 応答時間はそのままスループットは向上

実行時間

- プログラムを実行したときに掛かる時間
 - ユーザの書いたプログラムそのものの処理
 - 加減乗除、比較、分岐等
 - メモリアクセス時間
 - メモリへの書き込み、メモリからの読み出し
 - I/O時間
 - ファイルの読み込み、書き出し
 - OSによるオーバーヘッド
 - タスクスケジューリング、メモリ保護
 - 他のタスクの処理時間
 - CPUを独占できるわけではない（組込みシステムを除く）

実行時間

- とあるプログラムの実行時間



CPUクロック

- CPUはクロック（基準となる周期信号）に同期して動作する
 - クロックサイクル時間
 - クロックの間隔
 - クロックサイクル時間が1sなら、1秒に1回動作する（1秒に1命令とは限らない）
 - クロック周期の表し方
 - クロックサイクル時間(s)
 - クロック周期(Hz)
(逆数の関係)
クロックサイクル時間 = 250ps
クロック周波数 = 1/250ps = 4GHz

CPU性能

$$\begin{aligned} \text{あるプログラムのCPU実行時間} &= \frac{\text{そのプログラムのCPUクロックサイクル数} \times \text{クロックサイクル時間}}{\text{クロック周波数}} \\ &= \frac{\text{そのプログラムのCPUクロック} \cdot \text{サイクル数}}{\text{クロック周波数}} \end{aligned}$$

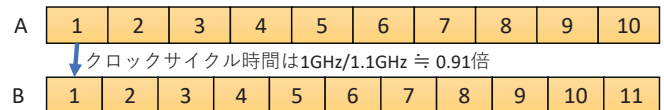
- CPU実行時間を短くする
 - クロックサイクル時間を短くする（ハードウェア）
 - プログラムが必要とするクロックサイクル数を減らす（ソフトウェア）

クロック周波数とクロックサイクル数

- コンピュータA (1GHz)
 - 積和命令(MAD : Multiply-Add) を持つてる
 - MAD A, B, C → 1サイクルで実行
 - $A = A + B * C$
 - コンピュータB (1.1GHz)
 - MAD命令を持ってない
 - MAD A, B, C
 - MULT T, B, C
 - ADD A, A, B
- MAC(Multiply-Accumulate)命令とも呼ぶ
- 2サイクルに分割
- 複雑な(MAD)命令を除いたため、CPUのクロック周波数は向上したが、プログラムの命令数は増加したため、クロックサイクル数は増加

CPU実行時間(1/3)

- コンピュータA (1GHz)
 - 積和命令(MAD : Multiply-Add) を持つてる
- コンピュータB (1.1GHz)
 - MAD命令を持ってない

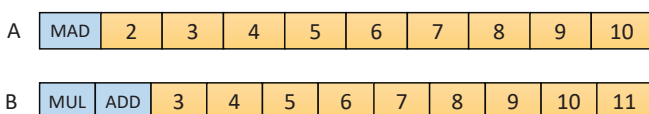


プログラムが同じクロックサイクル数なら性能は1.1倍

もし、最初の命令が積和演算(MAD)なら？

CPU実行時間(2/3)

- コンピュータA (1GHz)
 - 積和命令(MAD : Multiply-Add) を持つてる
- コンピュータB (1.1GHz)
 - MAD命令を持ってない

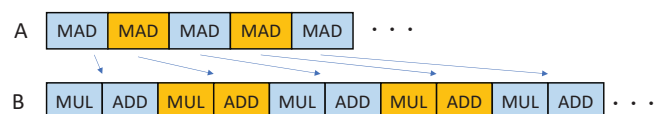


CPU実行時間は同じ

もし、すべての命令が積和演算(MAD)なら？

CPU実行時間(3/3)

- コンピュータA (1GHz)
 - 積和命令(MAD : Multiply-Add) を持つてる
- コンピュータB (1.1GHz)
 - MAD命令を持ってない



コンピュータAの方が圧倒的に早くなる

CPU実行時間は、クロック周波数とプログラム中の命令の種類や頻度に依存

命令の性能

- 命令によって必要なサイクル数は変わる
 - 典型的なMIPSの場合
 - 加減算：1サイクル
 - 乗算：10サイクル
 - 除算：69サイクル
- プログラムのCPUクロックサイクル数：

$$\text{プログラムの実行命令数} \times \text{命令当たりの平均クロックサイクル数}$$

CPI (Clock cycle per instruction) と呼ぶ

CPI (Clock cycle per instruction)

- 命令の性能

$$\text{CPUクロックサイクル数} = \frac{\text{プログラムの実行命令数}}{\text{CPI}}$$

$$\text{CPI} = \frac{\text{CPUクロックサイクル数}}{\text{プログラムの実行命令数}}$$

IPC (Instruction per clock cycle)

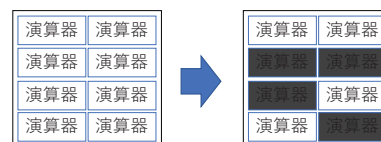
- プロセッサの性能を表す場合は、IPCが用いられることが多い

$$\text{IPC} = \frac{1}{\text{CPI}}$$

- IPCはCPIの逆数（1クロックで何命令実行できるかを表す）
- 最近のコンピュータは1クロックで数命令同時に実行できるので、IPCは3や4（1以上）になる

ダークシリコン問題

- プロセッサの消費電力の大部分は熱
- ムーアの法則に従い、1チップに搭載できるトランジスタ数は大幅に増加
 - ⇒チップ全体に電力供給できなくなる



電気・電力・エネルギー

- 電圧(V)
 - 2点間の電位差
- 電流(I)
 - 単位時間あたりに流れる電子の量
- 電力(P)
 - 電圧と電流の積($P=VI$)
- 電力量/エネルギー(E)
 - あるタスクを実行するのに必要な電力
 - $E=P \times T$ (Tは時間)

消費電力

- 現代のデジタル回路(含むCPU)はCMOS
 - PMOS、NMOSの2種類のトランジスタ（スイッチ）を組み合わせる回路の総称
 - MOSトランジスタの入力はキャパシタ（コンデンサ）と等価
 - CPUの消費電力(P)
 - $\text{電力}_{CPU} \propto \frac{1}{2} \times \text{容量性負荷} \times \text{電圧}^2 \times \text{周波数}$
 - CPUの消費エネルギー($E=P \times T$)
 - 100MHzで20分の消費エネルギー
 - 200MHzで10分の消費エネルギー } 同じ？

単体プロセッサからマルチプロセッサへ

- プロセッサの性能を簡単に上げるにはクロック周波数を上げればよい
 - ただし、クロック周波数を上げるためには電源電圧を上げる必要がある
 - $\text{電力}_{CPU} \propto \frac{1}{2} \times \text{容量性負荷} \times \text{電圧}^2 \times \text{周波数}$
 - 消費電力は電圧の2乗で増加
- ⇒タスクを並列化し、マルチプロセッサで実行する

アムダール(Amdahl)の法則(1/2)

- 並列化すれば万事解決？
 - 1CPUで100時間掛かるタスク
 - CPUを100個用意したら実行時間は？
 - $100[\text{時間/CPU}] / 100[\text{CPU}] = 1\text{時間}??$
- ⇒すべてを並列化できるなら真
- 並列化できない部分があると???
- プログラム中の $P(0 \leq P \leq 1)$ だけ並列化できるとする。プロセッサが無限にあるとすると、実行時間(T)は

$$T = \lim_{n \rightarrow \infty} \frac{1}{n} + (1 - P)$$

アムダール(Amdahl)の法則(2/2)

- $P=0.8$ （プログラムの8割が並列化可）とする。
1CPUで100時間掛かるプログラムの実行時間はどれだけ減る？

プロセッサ数	時間
1	100
2	60
3	47
4	40
∞	20

- 高速化は並列化できる部分の割合に依存

⇒並列化できる部分は半分ならどう頑張っても性能は2倍にしかない

ベンチマーク

- CPUの性能は、
 - アーキテクチャ
 - クロック周波数
 - プログラム（アルゴリズム含）
 - データ等に依存する。
- では、コンピュータAとコンピュータBの性能を比較したい場合は？
 - ベンチマークプログラムを用いる
 - プログラムとデータセット
 - SPEC, MiBench, Linpack等、様々
 - 3DMark, CrystalDiskMark, FFベンチ（Win系）

SPECpower

- CPUの性能を決める重要な要素
 - 処理速度だけでいい？
- もう一つの重要な要素
 - 消費エネルギー
- 電力測定用のベンチマーク
 - SPECpower
- 電力性能比（ワット当たりの性能）も重要な要素になっている